INF461 - PRINCIPES ET PROCESSUS DU LOGICIEL

Rapport intégré
Patrons de création

Étude comparative de quatre patrons, dans l'ordre demandé

Étudiant : Theodore Wilfried Bitjong

Matricule : 23U2331

Organisation GitHub :
https://github.com/inf461-bitjong

11 novembre 2026

Sommaire

1. Factory Method
2. Abstract Factory
3. Builder
4. Singleton

Chaque partie présente le patron, l'exemple étudié en cours, son implémentation et une variante concrète. Les diagrammes associés sont conservés dans leurs dossiers d'origine.

INF461 - PATRONS DE CRÉATION

Factory Method

Étude de l'exemple du cours et de sa variante

Dépôt GitHub : https://github.com/inf461-bitjong/factory-method

Theodore Wilfried Bitjong - 23U2331

1. Présentation du patron

Factory Method (fabrique) est un patron de création qui définit une méthode de création permettant aux sous-classes de choisir le type concret de l'objet produit. Le client manipule une abstraction et délègue l'instanciation à une fabrique, plutôt que de construire lui-même chaque produit concret.

Structure générique

- **Produit abstrait :** définit l'interface commune des objets créés.
- **Produits concrets :** implémentent cette interface avec des comportements spécialisés.
- **Créateur abstrait :** expose la méthode de création, abstraite ou appelée par une méthode commune.
- **Créateurs concrets :** redéfinissent la méthode de création pour instancier le produit correspondant.

> **Choix du modèle générique.** Le patron Factory Method convient aux deux exemples, car chacun propose plusieurs produits d'une même famille et laisse une fabrique concrète décider du produit instancié. Le client utilise ensuite le type abstrait, sans dépendre des opérations propres à chaque produit.

Application dans ce projet

Dans l'exemple du cours, `ProduitFactory` fournit `getProduit()`, qui délègue à `createProduit()`, redéfinie dans les fabriques concrètes. Dans la variante, `NotificationSenderFactory` déclare directement `createSender()`, redéfinie par les fabriques SMS, Email et Push. Dans les deux cas, le client sélectionne explicitement la fabrique concrète utilisée.

---

2. Exemple vu en cours

L'exemple présente une famille de produits abstraits `Produit`. Les classes `ProduitA`, `ProduitB` et `ProduitC` implémentent la même opération. Le créateur abstrait `ProduitFactory` définit `getProduit()`, qui appelle sa méthode de fabrique protégée `createProduit()`. Chaque fabrique concrète crée un type de produit différent.

Organisation de la solution

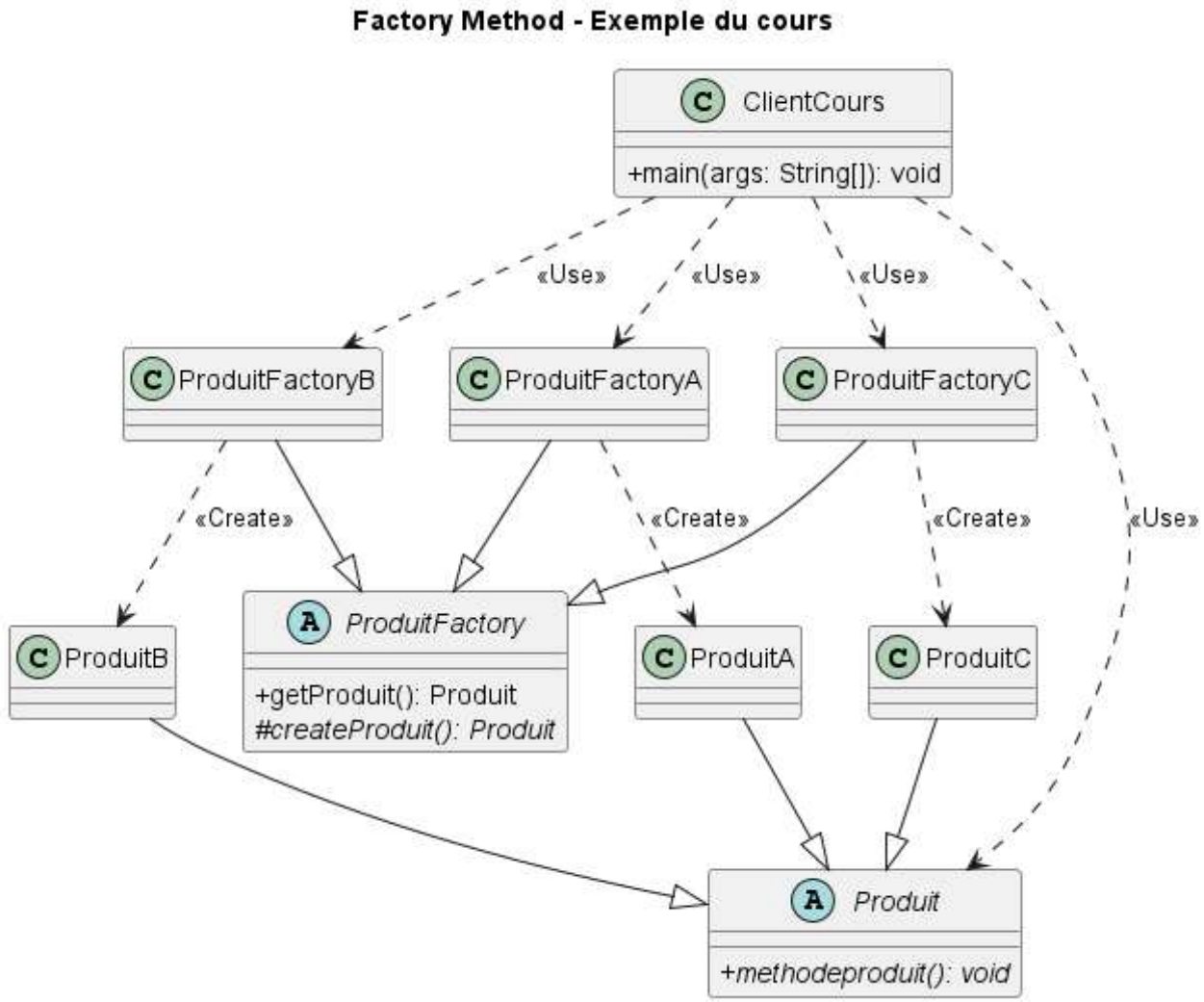


Figure 1 — Organisation des produits et fabriques dans l'exemple du cours.

3. Implémentation de l'exemple du cours

Les extraits suivants reprennent le code source du dépôt.

ClientCours.java

```
package cours;

import cours.abstractions.Produit;
import cours.abstractions.ProduitFactory;
import cours.factories.ProduitFactoryA;
import cours.factories.ProduitFactoryB;
import cours.factories.ProduitFactoryC;

public class ClientCours {
    public static void main(String[] args) {
        System.out.println("--- EXEMPLE DU COURS : Factory Method ---");

        ProduitFactory factoryA = new ProduitFactoryA();
        ProduitFactory factoryB = new ProduitFactoryB();
        ProduitFactory factoryC = new ProduitFactoryC();

        Produit produit = factoryA.getProduit();
        produit.methodeproduit();

        produit = factoryB.getProduit();
        produit.methodeproduit();

        produit = factoryC.getProduit();
        produit.methodeproduit();
    }
}
```

Produit.java

```
package cours.abstractions;

public abstract class Produit {
    public abstract void methodeproduit();
}
```

ProduitFactory.java

```
package cours.abstractions;

public abstract class ProduitFactory {
    public Produit getProduit() {
        return createProduit();
    }

    protected abstract Produit createProduit();
}
```

ProduitFactoryA.java

```
package cours.factories;

import cours.abstractions.Produit;
import cours.abstractions.ProduitFactory;
import cours.models.ProduitA;

public class ProduitFactoryA extends ProduitFactory {
    @Override
    protected Produit createProduit() {
        return new ProduitA();
    }
}
```

ProduitFactoryB.java

```
package cours.factories;

import cours.abstractions.Produit;
import cours.abstractions.ProduitFactory;
import cours.models.ProduitB;

public class ProduitFactoryB extends ProduitFactory {
    @Override
    protected Produit createProduit() {
        return new ProduitB();
    }
}
```

ProduitFactoryC.java

```
package cours.factories;

import cours.abstractions.Produit;
import cours.abstractions.ProduitFactory;
import cours.models.ProduitC;

public class ProduitFactoryC extends ProduitFactory {
    @Override
    protected Produit createProduit() {
        return new ProduitC();
    }
}
```

ProduitA.java

```
package cours.models;

import cours.abstractions.Produit;

public class ProduitA extends Produit {
    @Override
    public void methodeproduit() {
        System.out.println("je suis un produit de type A");
        System.out.println("produitA.methodeproduit()");
    }
}
```

ProduitB.java

```
package cours.models;

import cours.abstractions.Produit;

public class ProduitB extends Produit {
    @Override
    public void methodeproduit() {
        System.out.println("je suis un produit de type B");
        System.out.println("produitB.methodeproduit()");
    }
}
```

ProduitC.java

```
package cours.models;

import cours.abstractions.Produit;

public class ProduitC extends Produit {
    @Override
    public void methodeproduit() {
        System.out.println("je suis un produit de type C");
        System.out.println("produitC.methodeproduit()");
    }
}
```

4. Exemple modifié : notifications

La variante applique le patron à l'envoi de notifications. `NotificationSender` est l'abstraction des expéditeurs ; les classes `EmailSender`, `SmsSender` et `PushSender` fournissent chacune leur comportement `send()`. Les fabriques concrètes créent l'expéditeur associé au canal demandé.

Organisation de la solution

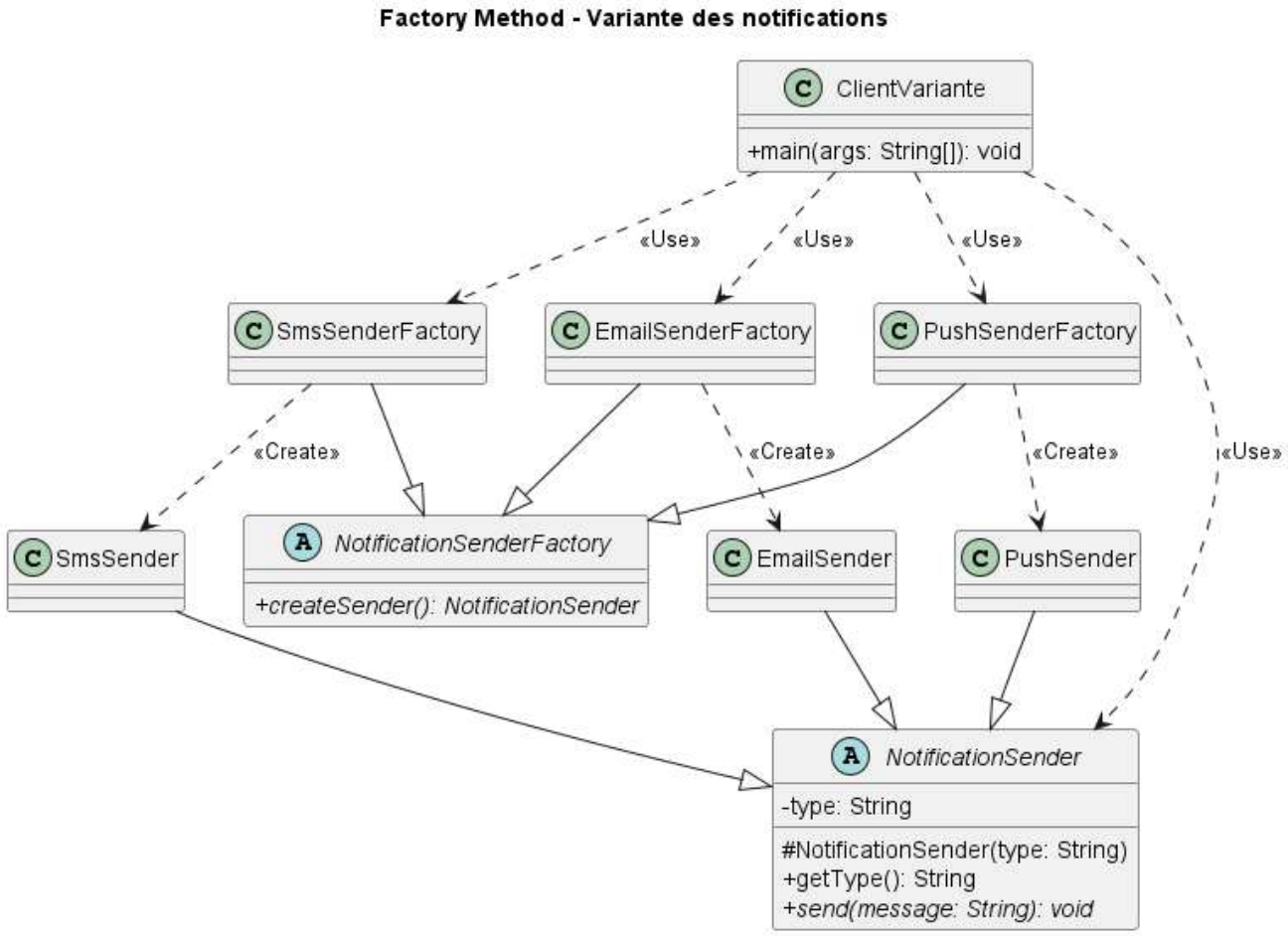


Figure 2 — Organisation des expéditeurs et fabriques pour les notifications.

Analyse du choix

Factory Method est adapté, car les canaux de notification constituent une famille de produits substituables derrière l'abstraction `NotificationSender`. Une nouvelle implémentation peut ajouter un expéditeur et sa fabrique sans modifier les classes déjà présentes. Le client de cette démonstration choisit toutefois chaque fabrique concrète dans son code ; la sélection du canal n'est donc pas encore externalisée.

> **Portée de la démonstration :** les implémentations affichent le message sur la sortie standard. Elles illustrent la création polymorphique, mais n'effectuent pas de véritable envoi par courriel, SMS ou notification push.

5. Implémentation de la variante

Les extraits suivants reprennent le code source du dépôt.

ClientVariante.java

```
package variante;

import variante.abstractions.NotificationSenderFactory;
import variante.abstractions.NotificationSender;
import variante.factories.EmailSenderFactory;
import variante.factories.PushSenderFactory;
import variante.factories.SmsSenderFactory;

public class ClientVariante {
    public static void main(String[] args) {
        System.out.println("--- EXEMPLE MODIFIE : Factory de Senders de Notification ---");

        NotificationSenderFactory factorySms = new SmsSenderFactory();
        NotificationSender smsSender = factorySms.createSender();
        smsSender.send("Votre code de validation est 9832");

        NotificationSenderFactory factoryEmail = new EmailSenderFactory();
        NotificationSender emailSender = factoryEmail.createSender();
        emailSender.send("Bienvenue sur notre application !");

        NotificationSenderFactory factoryPush = new PushSenderFactory();
        NotificationSender pushSender = factoryPush.createSender();
        pushSender.send("Vous avez un nouveau message.");
    }
}
```

NotificationSender.java

```
package variante.abstractions;

public abstract class NotificationSender {
    private final String type;

    protected NotificationSender(String type) {
        this.type = type;
    }

    public String getType() {
        return type;
    }

    public abstract void send(String message);
}
```

NotificationSenderFactory.java

```
package variante.abstractions;

public abstract class NotificationSenderFactory {
    public abstract NotificationSender createSender();
}
```

EmailSenderFactory.java

```
package variante.factories;

import variante.abstractions.NotificationSenderFactory;
import variante.abstractions.NotificationSender;
import variante.models.EmailSender;

public class EmailSenderFactory extends NotificationSenderFactory {
    @Override
    public NotificationSender createSender() {
        return new EmailSender();
    }
}
```

SmsSenderFactory.java

```
package variante.factories;

import variante.abstractions.NotificationSenderFactory;
import variante.abstractions.NotificationSender;
import variante.models.SmsSender;

public class SmsSenderFactory extends NotificationSenderFactory {
    @Override
    public NotificationSender createSender() {
        return new SmsSender();
    }
}
```

PushSenderFactory.java

```
package variante.factories;

import variante.abstractions.NotificationSenderFactory;
import variante.abstractions.NotificationSender;
import variante.models.PushSender;

public class PushSenderFactory extends NotificationSenderFactory {
    @Override
    public NotificationSender createSender() {
        return new PushSender();
    }
}
```

EmailSender.java

```
package variante.models;

import variante.abstractions.NotificationSender;

public class EmailSender extends NotificationSender {
    public EmailSender() {
        super("EMAIL");
    }

    @Override
    public void send(String message) {
        System.out.println("[" + getType() + "] Envoi d'un courriel : " + message);
    }
}
```

SmsSender.java

```
package variante.models;

import variante.abstractions.NotificationSender;

public class SmsSender extends NotificationSender {
    public SmsSender() {
        super("SMS");
    }

    @Override
    public void send(String message) {
        System.out.println("[" + getType() + "] Envoi d'un SMS : " + message);
    }
}
```

PushSender.java

```
package variante.models;

import variante.abstractions.NotificationSender;

public class PushSender extends NotificationSender {
    public PushSender() {
        super("PUSH");
    }

    @Override
    public void send(String message) {
        System.out.println("[" + getType() + "] Envoi d'une notification push : " + message);
    }
}
```

INF461 - PATRONS DE CRÉATION

Abstract Factory

Étude de l'exemple du cours et de sa variante

Dépôt GitHub : https://github.com/inf461-bitjong/abstact-factory

Theodore Wilfried Bitjong - 23U2331

1. Présentation du patron

Abstract Factory (Fabrique abstraite) est un patron de création qui fournit une interface pour créer plusieurs produits liés, sans exposer leurs classes concrètes. Le client choisit une fabrique concrète et utilise les produits retournés au travers de leurs abstractions.

Structure générique

- **Fabrique abstraite :** déclare une opération de création pour chaque type de produit de la famille.
- **Fabriques concrètes :** créent les produits concrets qui composent chacune des familles.
- **Produits abstraits et concrets :** définissent les interfaces communes et leurs variantes.
- **Client :** dépend des abstractions et emploie ensemble les produits d'une même fabrique.

> **Choix du modèle générique.** Le besoin porte sur la création cohérente de plusieurs types de produits. Dans l'exemple du cours, chaque fabrique produit un ProduitA et un ProduitB appartenant à la même famille. Dans la variante, chaque fabrique de thème crée un bouton et un menu assortis. Abstract Factory évite au client de connaître les classes concrètes et facilite l'ajout d'une famille complète.

Intérêt et limite

Le changement de famille se fait en choisissant une autre fabrique concrète, tandis que le code client reste fondé sur les mêmes abstractions. En contrepartie, l'ajout d'un nouveau type de produit nécessite de modifier l'interface de fabrique et chacune de ses implémentations.

2. Exemple vu en cours

L'exemple présente deux familles de produits. `ProduitFactory1` crée `ProduitA1` et `ProduitB1`, tandis que `ProduitFactory2` crée `ProduitA2` et `ProduitB2`. Le client appelle les méthodes des abstractions sans dépendre des implémentations concrètes.

Modélisation

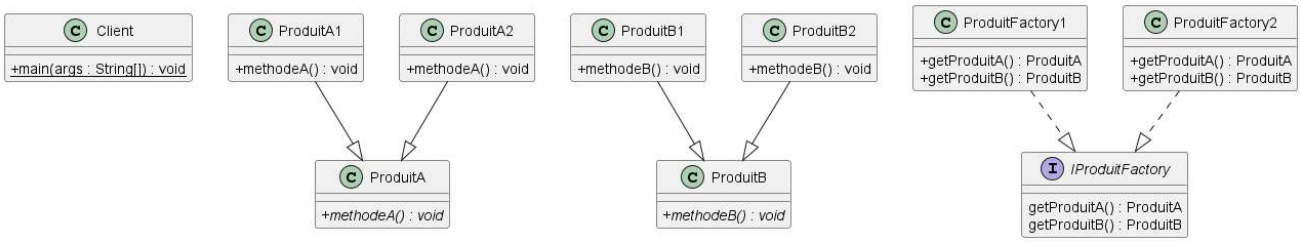


Figure 1 — Diagramme de classe de l'exemple du cours.

> **Lecture du diagramme :** les deux fabriques concrètes implémentent `IProduitFactory`. Elles créent chacune un produit concret de type A et un produit concret de type B. Les classes concrètes héritent de leurs produits abstraits respectifs.

Comportement du client

`Client` instancie successivement les deux fabriques, récupère les deux produits de chaque famille, puis appelle `methodeA()` et `methodeB()`. La sortie montre les implémentations qui ont été sélectionnées.

3. Implémentation de l'exemple du cours

Abstractions — IProduitFactory.java, ProduitA.java et ProduitB.java

```
package cours.abstractions;

public interface IProduitFactory {
    ProduitA getProduitA();
    ProduitB getProduitB();
}

public abstract class ProduitA {
    public abstract void methodeA();
}

public abstract class ProduitB {
    public abstract void methodeB();
}
```

Fabriques — ProduitFactory1.java et ProduitFactory2.java

```
package cours.factories;

public class ProduitFactory1 implements IProduitFactory {
    @Override
    public ProduitA getProduitA() {
        return new ProduitA1();
    }

    @Override
    public ProduitB getProduitB() {
        return new ProduitB1();
    }
}

public class ProduitFactory2 implements IProduitFactory {
    @Override
    public ProduitA getProduitA() {
        return new ProduitA2();
    }

    @Override
    public ProduitB getProduitB() {
        return new ProduitB2();
    }
}
```

Les imports des classes de chaque fichier sont omis dans ces extraits ; les méthodes et relations montrées correspondent au code source du dépôt.

3. Implémentation de l'exemple du cours

Produits concrets — modèles A et B

```
package cours.models;
import cours.abstractions.ProduitA;

public class ProduitA1 extends ProduitA {
    @Override
    public void methodeA() {
        System.out.println("ProduitA1.methodeA()");
    }
}

public class ProduitA2 extends ProduitA {
    @Override
    public void methodeA() {
        System.out.println("ProduitA2.methodeA()");
    }
}

package cours.models;
import cours.abstractions.ProduitB;

public class ProduitB1 extends ProduitB {
    @Override
    public void methodeB() {
        System.out.println("ProduitB1.methodeB()");
    }
}

public class ProduitB2 extends ProduitB {
    @Override
    public void methodeB() {
        System.out.println("ProduitB2.methodeB()");
    }
}
```

Client.java

```
package cours;

public class Client {
    public static void main(String[] args) {
        IProduitFactory produitFactory1 = new ProduitFactory1();
        IProduitFactory produitFactory2 = new ProduitFactory2();

        System.out.println("--- Utilisation de la premiere fabrique (Famille 1) ---");
        ProduitA produitA = produitFactory1.getProduitA();
        ProduitB produitB = produitFactory1.getProduitB();
        produitA.methodeA();
        produitB.methodeB();

        System.out.println("\n--- Utilisation de la seconde fabrique (Famille 2) ---");
        produitA = produitFactory2.getProduitA();
        produitB = produitFactory2.getProduitB();
        produitA.methodeA();
        produitB.methodeB();
    }
}
```

Les imports des classes utilisées dans les fichiers sont omis uniquement pour condenser la présentation.

4. Exemple modifié : thèmes d'interface

La variante applique le patron à deux thèmes d'interface, sombre et clair. Chaque thème fournit une implémentation de `Bouton` et de `Menu`. `ClientVariante` construit les deux familles et invoque leurs opérations d'affichage.

Modélisation

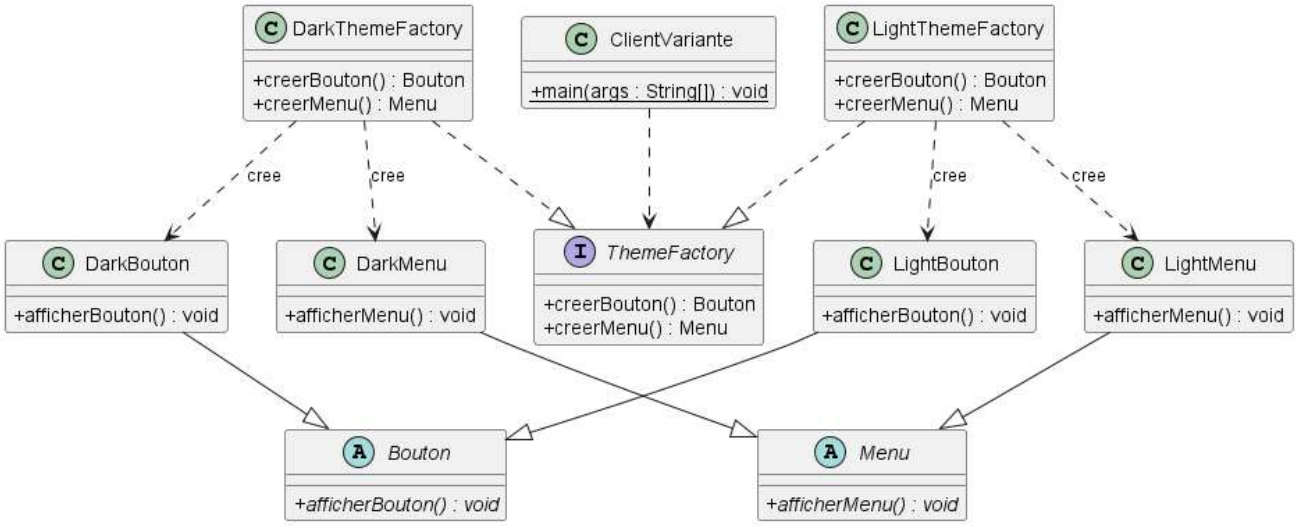


Figure 2 — Diagramme de classe des fabriques de thèmes et des composants d'interface.

Analyse du choix

Un thème est une famille de composants qui doivent présenter un style cohérent. `DarkThemeFactory` associe `DarkBouton` à `DarkMenu` et `LightThemeFactory` associe les composants clairs. Le client dépend de `ThemeFactory`, `Bouton` et `Menu` plutôt que des classes concrètes ; il peut donc changer de famille en substituant la fabrique.

> **Résultat attendu :** le programme affiche les messages du bouton et du menu du thème sombre, puis ceux du thème clair. Cette variante illustre la création de produits assortis ; le rendu est ici simulé par des messages dans la console.

5. Implémentation de la variante

Abstractions et fabrique — ThemeFactory, Bouton et Menu

```
package variante.abstractions;

public interface ThemeFactory {
    Bouton creerBouton();
    Menu creerMenu();
}

public abstract class Bouton {
    public abstract void afficherBouton();
}

public abstract class Menu {
    public abstract void afficherMenu();
}
```

Fabriques concrètes — DarkThemeFactory et LightThemeFactory

```
package variante.factories;

public class DarkThemeFactory implements ThemeFactory {
    @Override
    public Bouton creerBouton() {
        return new DarkBouton();
    }

    @Override
    public Menu creerMenu() {
        return new DarkMenu();
    }
}

public class LightThemeFactory implements ThemeFactory {
    @Override
    public Bouton creerBouton() {
        return new LightBouton();
    }

    @Override
    public Menu creerMenu() {
        return new LightMenu();
    }
}
```

Les imports de chaque fichier sont omis ; les extraits regroupent les types par rôle pour faciliter la lecture.

5. Implémentation de la variante

Produits concrets — boutons et menus

```
package variante.models;

public class DarkBouton extends Bouton {
    @Override
    public void afficherBouton() {
        System.out.println("[DARK MODE] Affichage d'un bouton sombre (fond gris fonce, texte blanc).");
    }
}

public class DarkMenu extends Menu {
    @Override
    public void afficherMenu() {
        System.out.println("[DARK MODE] Affichage d'un menu deroulant sombre.");
    }
}

public class LightBouton extends Bouton {
    @Override
    public void afficherBouton() {
        System.out.println("[LIGHT MODE] Affichage d'un bouton lumineux (fond blanc, texte noir).");
    }
}

public class LightMenu extends Menu {
    @Override
    public void afficherMenu() {
        System.out.println("[LIGHT MODE] Affichage d'un menu deroulant lumineux.");
    }
}
```

ClientVariante.java

```
package variante;

public class ClientVariante {
    public static void main(String[] args) {
        System.out.println("=== EXEMPLE MODIFIE : Abstract Factory (Themes UI) ===");

        ThemeFactory darkFactory = new DarkThemeFactory();
        Bouton darkBouton = darkFactory.creerBouton();
        Menu darkMenu = darkFactory.creerMenu();
        System.out.println("\nChargement du theme sombre :");
        darkBouton.afficherBouton();
        darkMenu.afficherMenu();

        ThemeFactory lightFactory = new LightThemeFactory();
        Bouton lightBouton = lightFactory.creerBouton();
        Menu lightMenu = lightFactory.creerMenu();
        System.out.println("\nChargement du theme clair :");
        lightBouton.afficherBouton();
        lightMenu.afficherMenu();
    }
}
```

Les imports de chaque fichier sont omis ; les méthodes et messages affichés sont ceux du code source du dépôt.

INF461 - PATRONS DE CRÉATION

Builder

Étude de l'exemple du cours et de sa variante

Dépôt GitHub : https://github.com/inf461-bitjong/builder

Theodore Wilfried Bitjong - 23U2331

1. Présentation du patron

Builder est un patron de création qui sépare la construction d'un objet complexe de sa représentation. Un client choisit un monteur concret et lui demande de construire le produit ; les étapes et les valeurs assemblées peuvent ainsi varier sans changer le produit final ni le code client.

Rôles du modèle générique

Rôle	Responsabilité
Produit	Objet construit et remis au client.
Builder abstrait	Définit l'accès au produit et les opérations de construction disponibles.
Builder concret	Réalise les opérations et configure le produit.
Directeur ou configurateur	Orchestre les étapes ou fournit des configurations prédéfinies.
Client	Demande la construction et utilise le produit obtenu.

> **Choix du modèle générique.** Dans l'exemple du cours, une pizza est assemblée en plusieurs étapes qui dépendent de la recette. Dans la variante, un ordinateur est configuré avec plusieurs caractéristiques indépendantes, et les méthodes chaînables du builder rendent ces configurations lisibles. Dans les deux cas, Builder isole la construction du produit de son utilisation.

Exemples présentés

- **Cours :** `Directeur` choisit un monteur de pizza et retourne la pizza terminée.
- **Variante :** `Configurator` prépare des profils de jeu et de bureau au moyen d'un builder fluide.

2. Exemple vu en cours : les pizzas

Le produit `Pizza` contient une pâte, une sauce et une garniture. La classe abstraite `MonteurPizza` définit les étapes de construction et les exécute dans `getPizza()`. Les monteurs concrets fournissent les valeurs propres aux pizzas Reine et Piquante. `Directeur` sélectionne le monteur correspondant.

Modélisation

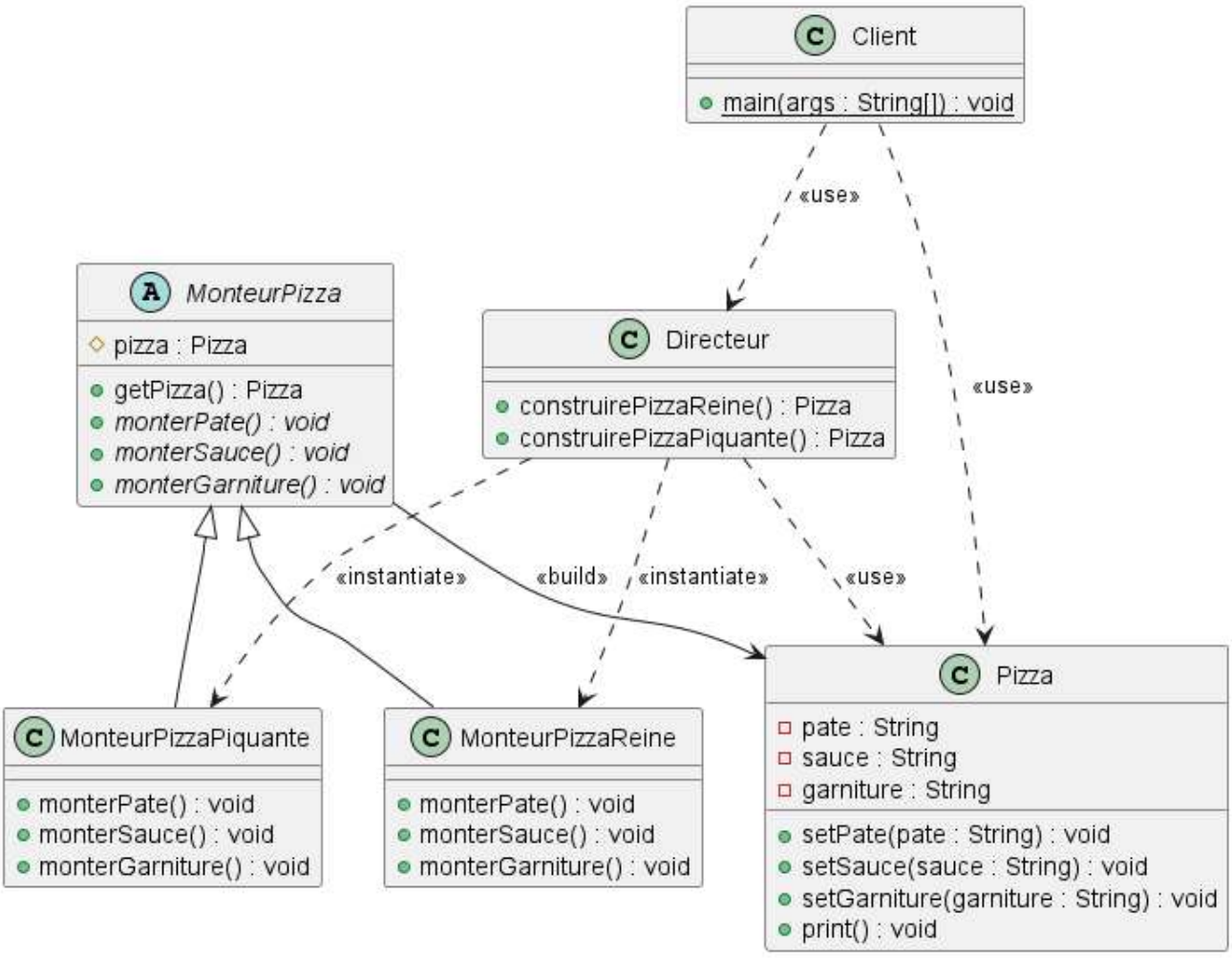


Figure 1 — Diagramme de classes de l'exemple du cours.

> **Analyse.** Le modèle classique avec un directeur, un builder abstrait, des builders concrets et un produit convient ici parce que les étapes (pâte, sauce, garniture) sont communes, tandis que leur résultat change selon la recette.

3. Implémentation de l'exemple du cours

Client.java

```
package cours;

public class Client {
    public static void main(String[] args) {
        Directeur directeur = new Directeur();

        System.out.println(".....Pizza Reine Fabriquée.....");
        directeur.construirePizzaReine().print();

        System.out.println("\n...Pizza Piquante Fabriquée...");
        directeur.construirePizzaPiquante().print();
    }
}
```

Directeur.java

```
package cours;

import cours.abstractions.MonteurPizza;
import cours.builders.MonteurPizzaPiquante;
import cours.builders.MonteurPizzaReine;
import cours.modeles.Pizza;

public class Directeur {
    public Pizza construirePizzaReine() {
        MonteurPizza pizzaReine = new MonteurPizzaReine();
        return pizzaReine.getPizza();
    }

    public Pizza construirePizzaPiquante() {
        MonteurPizza pizzaPiquante = new MonteurPizzaPiquante();
        return pizzaPiquante.getPizza();
    }
}
```

3. Implémentation de l'exemple du cours

MonteurPizza.java

```
package cours.abstractions;

import cours.modeles.Pizza;

public abstract class MonteurPizza {
    protected Pizza pizza = new Pizza();

    public Pizza getPizza() {
        monterPate();
        monterSauce();
        monterGarniture();
        return pizza;
    }

    public abstract void monterPate();

    public abstract void monterSauce();

    public abstract void monterGarniture();
}
```

MonteurPizzaReine.java

```
package cours.builders;

import cours.abstractions.MonteurPizza;

public class MonteurPizzaReine extends MonteurPizza {
    @Override
    public void monterPate() {
        pizza.setPate("croisée");
    }

    @Override
    public void monterSauce() {
        pizza.setSauce("douce");
    }

    @Override
    public void monterGarniture() {
        pizza.setGarniture("jambon+champignon");
    }
}
```

3. Implémentation de l'exemple du cours

MonteurPizzaPiquante.java

```
package cours.builders;

import cours.abstractions.MonteurPizza;

public class MonteurPizzaPiquante extends MonteurPizza {
    @Override
    public void monterPate() {
        pizza.setPate("feuilletée");
    }

    @Override
    public void monterSauce() {
        pizza.setSauce("piquante");
    }

    @Override
    public void monterGarniture() {
        pizza.setGarniture("pepperoni+salami");
    }
}
```

Pizza.java

```
package cours.modeles;

public class Pizza {
    private String pate = "";
    private String sauce = "";
    private String garniture = "";

    public void setPate(String pate) {
        this.pate = pate;
    }

    public void setSauce(String sauce) {
        this.sauce = sauce;
    }

    public void setGarniture(String garniture) {
        this.garniture = garniture;
    }

    public void print() {
        System.out.println("Pate = " + this.pate + "\nSauce = " + this.sauce + "\nGarniture = " + this.garniture);
    }
}
```

4. Exemple modifié : configuration d'ordinateurs

La variante remplace la recette de pizza par la configuration d'un ordinateur. `Computer` est le produit ; `ComputerBuilder` décrit les opérations de configuration ; `CustomComputerBuilder` applique les valeurs au produit et retourne le builder pour permettre l'enchaînement. `Configurator` propose deux configurations prédéfinies.

Modélisation

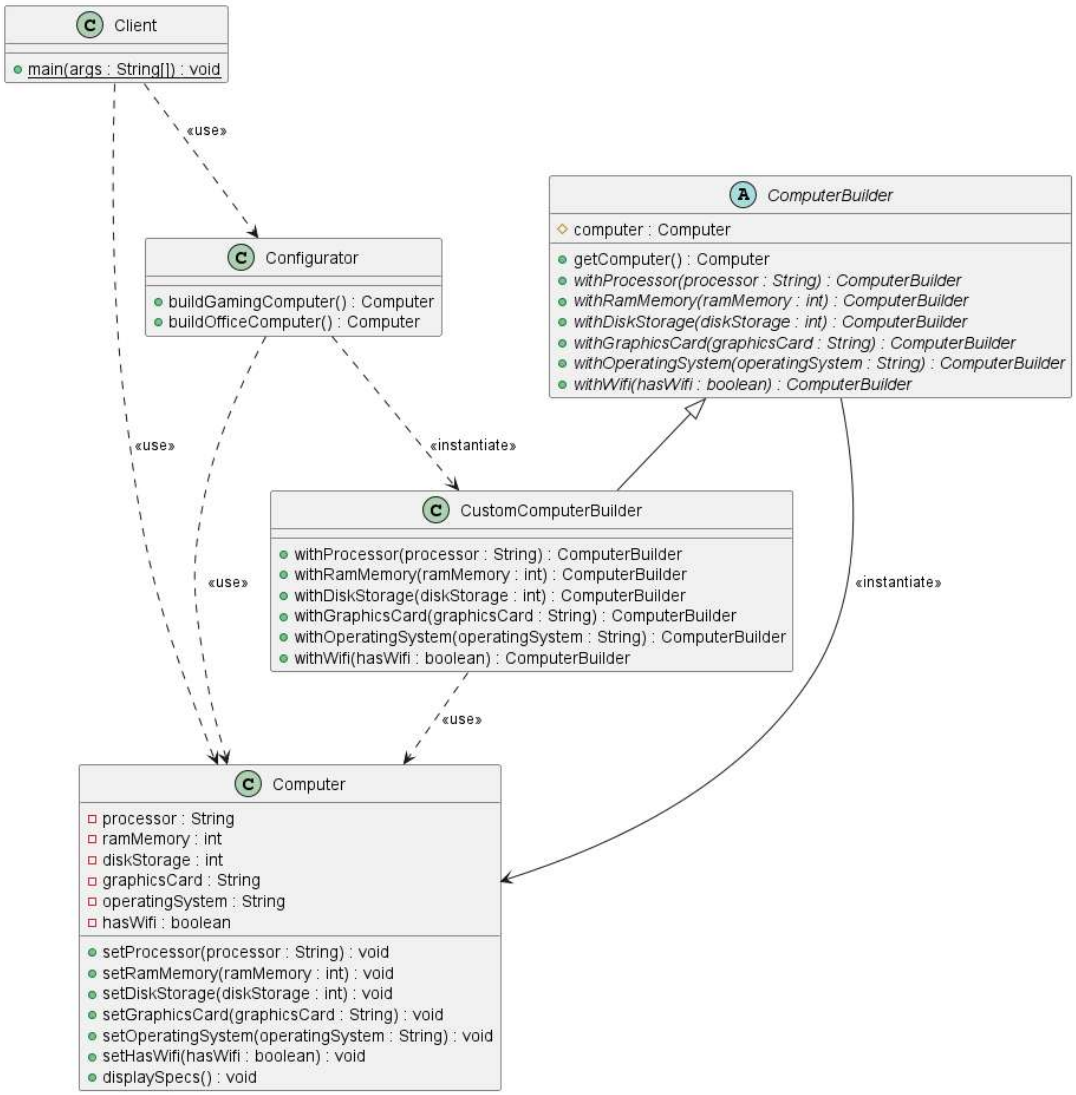


Figure 2 — Diagramme de classes de la variante pour les ordinateurs.

> **Analyse.** Le Builder fluide convient à un produit dont les paramètres sont nombreux et configurables séparément. Chaque appel `with...` modifie une caractéristique et retourne le builder ; les séquences de configuration restent lisibles et le même produit peut représenter plusieurs profils.

5. Implémentation de la variante

Client.java

```
package variante;

public class Client {
    public static void main(String[] args) {
        Configurator configurator = new Configurator();

        System.out.println("=== Configurator Gaming Computer ===");
        configurator.buildGamingComputer().displaySpecs();

        System.out.println("\n=== Configurator Office Computer ===");
        configurator.buildOfficeComputer().displaySpecs();
    }
}
```

Configurator.java

```
package variante;

import variante.builders.CustomComputerBuilder;
import variante.modeles.Computer;

public class Configurator {
    public Computer buildGamingComputer() {
        return new CustomComputerBuilder()
                .withProcessor("Intel Core i9-14900K")
                .withRamMemory(64)
                .withDiskStorage(2000)
                .withGraphicsCard("NVIDIA RTX 4090")
                .withOperatingSystem("Windows 11 Pro")
                .withWifi(true)
                .getComputer();
    }

    public Computer buildOfficeComputer() {
        return new CustomComputerBuilder()
                .withProcessor("Intel Core i5-13400")
                .withRamMemory(16)
                .withDiskStorage(512)
                .withGraphicsCard("Intel UHD Graphics")
                .withOperatingSystem("Ubuntu 24.04 LTS")
                .withWifi(false)
                .getComputer();
    }
}
```

5. Implémentation de la variante

ComputerBuilder.java

```
package variante.abstractions;

import variante.modeles.Computer;

public abstract class ComputerBuilder {
    protected Computer computer = new Computer();

    public Computer getComputer() {
        return computer;
    }

    public abstract ComputerBuilder withProcessor(String processor);

    public abstract ComputerBuilder withRamMemory(int ramMemory);

    public abstract ComputerBuilder withDiskStorage(int diskStorage);

    public abstract ComputerBuilder withGraphicsCard(String graphicsCard);

    public abstract ComputerBuilder withOperatingSystem(String operatingSystem);

    public abstract ComputerBuilder withWifi(boolean hasWifi);
}
```

CustomComputerBuilder.java

```
package variante.builders;

import variante.abstractions.ComputerBuilder;

public class CustomComputerBuilder extends ComputerBuilder {
    @Override
    public ComputerBuilder withProcessor(String processor) {
        computer.setProcessor(processor);
        return this;
    }

    @Override
    public ComputerBuilder withRamMemory(int ramMemory) {
        computer.setRamMemory(ramMemory);
        return this;
    }

    @Override
    public ComputerBuilder withDiskStorage(int diskStorage) {
        computer.setDiskStorage(diskStorage);
        return this;
    }

    @Override
    public ComputerBuilder withGraphicsCard(String graphicsCard) {
        computer.setGraphicsCard(graphicsCard);
        return this;
    }

    @Override
    public ComputerBuilder withOperatingSystem(String operatingSystem) {
        computer.setOperatingSystem(operatingSystem);
        return this;
    }

    @Override
    public ComputerBuilder withWifi(boolean hasWifi) {
        computer.setHasWifi(hasWifi);
        return this;
    }
}
```

5. Implémentation de la variante

Computer.java

```
package variante.modeles;

public class Computer {
    private String processor;
    private int ramMemory;
    private int diskStorage;
    private String graphicsCard;
    private String operatingSystem;
    private boolean hasWifi;

    public void setProcessor(String processor) {
        this.processor = processor;
    }

    public void setRamMemory(int ramMemory) {
        this.ramMemory = ramMemory;
    }

    public void setDiskStorage(int diskStorage) {
        this.diskStorage = diskStorage;
    }

    public void setGraphicsCard(String graphicsCard) {
        this.graphicsCard = graphicsCard;
    }

    public void setOperatingSystem(String operatingSystem) {
        this.operatingSystem = operatingSystem;
    }

    public void setHasWifi(boolean hasWifi) {
        this.hasWifi = hasWifi;
    }

    public void displaySpecs() {
        System.out.println("Computer Specifications:");
        System.out.println(" - Processor: " + processor);
        System.out.println(" - RAM: " + ramMemory + " GB");
        System.out.println(" - Storage: " + diskStorage + " GB");
        System.out.println(" - Graphics Card: " + graphicsCard);
        System.out.println(" - Operating System: " + operatingSystem);
        System.out.println(" - Wi-Fi: " + (hasWifi ? "Yes" : "No"));
    }
}
```

> **Exécution.** Le client demande successivement un ordinateur de jeu et un ordinateur de bureau. Dans chaque cas, le configurateur chaîne les options du builder puis récupère le `Computer` final avec `getComputer()`.

6. Dépôt et conclusion

Le dépôt contient les sources Java des deux exemples ainsi que leurs diagrammes de classes. Le code du cours et la variante sont organisés séparément sous `src/cours` et `src/variante`.

Organisation GitHub : https://github.com/inf461-bitjong

Dépôt Builder : https://github.com/inf461-bitjong/builder

> **Conclusion.** L'exemple du cours montre comment déléguer des étapes de construction communes à des monteurs spécialisés. La variante conserve la séparation entre le produit et sa création, mais emploie une API fluide adaptée au réglage de nombreuses options. Ces deux mises en œuvre illustrent comment Builder rend la construction explicite tout en laissant les détails au builder concret.

INF461 - PATRONS DE CRÉATION

Singleton

Étude de l'exemple du cours et de sa variante

Dépôt GitHub : https://github.com/inf461-bitjong/singleton

Theodore Wilfried Bitjong - 23U2331

1. Présentation du patron

Le Singleton est un patron de création qui garantit qu'une classe ne possède qu'une seule instance et fournit un point d'accès global à celle-ci. Il est adapté lorsqu'une ressource ou un service doit être partagé et centralisé, par exemple un journaliseur ou un gestionnaire de configuration.

Structure générique

- **Instance statique privée :** conserve l'unique objet créé.
- **Constructeur privé :** empêche les autres classes de créer directement des objets avec `new`.
- **Accesseur statique :** crée l'objet au premier appel puis renvoie toujours la même instance.

> **Choix du modèle générique.** Dans les deux exemples, le modèle Singleton est retenu car le besoin est l'unicité d'un objet accessible depuis plusieurs clients. L'exemple du cours illustre la structure de base et ses opérations ; la variante l'applique à un service transversal réel, le journal d'application.

Considération de concurrence

La création paresseuse non synchronisée de l'exemple du cours convient à une démonstration séquentielle, mais n'assure pas à elle seule l'unicité lors d'appels simultanés de plusieurs threads. La variante synchronise l'accesseur `getInstance()`, protégeant ainsi la création concurrente de l'instance.

---

2. Exemple vu en cours

La classe `cours.Singleton` conserve deux valeurs entières et expose deux accesseurs surchargés : l'un crée l'objet sans paramètres, l'autre initialise `x` et `y`. Elle fournit aussi des méthodes de somme et de moyenne, un affichage des valeurs et une protection contre le clonage.

Modélisation

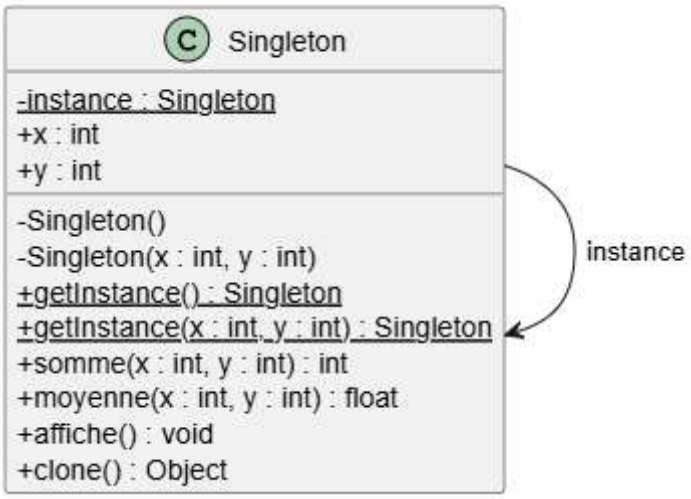


Figure 1 — Diagramme de classe de l'exemple du cours.

> **Comportement du test fourni :** `TestSingleton` appelle d'abord `getInstance()`, ce qui crée l'objet avec les valeurs par défaut (0, 0). Les appels suivants à `getInstance(8, 3)` et `getInstance(16, 19)` renvoient cette même instance ; les paramètres ne réinitialisent donc pas ses attributs.

---

3. Implémentation de l'exemple du cours

Singleton.java

```
package cours;

public final class Singleton implements Cloneable {
    private static Singleton instance = null;
    public int x;
    public int y;

    private Singleton() {
        super();
    }

    private Singleton(int x2, int y2) {
        super();
        this.x = x2;
        this.y = y2;
    }

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }

    public static Singleton getInstance(int x, int y) {
        if (instance == null) {
            instance = new Singleton(x, y);
        }
        return instance;
    }

    public int somme(int x, int y) {
        return x + y;
    }

    public float moyenne(int x, int y) {
        return (float) somme(x, y) / 2;
    }

    public void affiche() {
        System.out.println("\nJe suis une instance et mes valeurs sont : x = " + this.x + " et y = " + this.y);
    }

    @Override
    public Object clone() throws CloneNotSupportedException {
        throw new CloneNotSupportedException();
    }
}
```

TestSingleton.java

```
package cours;

public class TestSingleton {
    public static void main(String[] args) {
        int som = Singleton.getInstance().somme(2, 5);
        System.out.println("La somme est : " + som);

        Singleton s1 = Singleton.getInstance(8, 3);
        s1.affiche();

        Singleton s2 = Singleton.getInstance(16, 19);
        s2.affiche();
    }
}
```

4. Exemple modifié : AppLogger

La variante remplace l'objet de calcul par `variante.AppLogger`, un journaliseur partagé. Chaque module récupère le même service via `AppLogger.getInstance()`, puis écrit des messages informatifs ou d'erreur horodatés. L'initialisation est paresseuse ; l'accesseur synchronisé protège la création si plusieurs threads y accèdent en même temps. Le clonage est refusé.

Modélisation

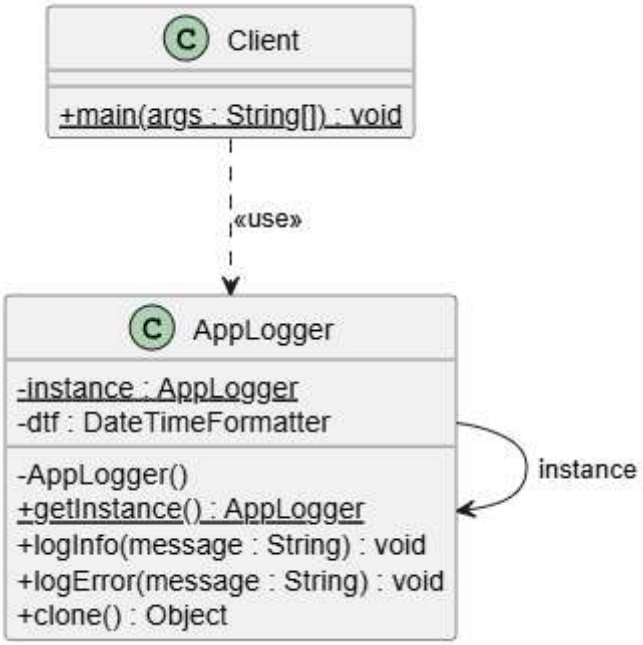


Figure 2 — Diagramme de classe de la variante AppLogger et de son client.

Analyse du choix

Un journaliseur est un service transversal utilisé par plusieurs parties d'une application. L'unique instance représente ici un point de journalisation central, évitant la création d'instances distinctes par chaque composant. Le client effectue deux demandes d'instance et compare les références ; le résultat attendu est `true`. Cette démonstration écrit sur la sortie standard et la sortie d'erreur, sans ouvrir de fichier de logs.

5. Implémentation de la variante

AppLogger.java

```
package variante;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public final class AppLogger {
    private static AppLogger instance = null;
    private DateTimeFormatter dtf;

    private AppLogger() {
        super();
        this.dtf = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
        System.out.println("[SYSTEM INIT] Initialisation du journal d'application unique (AppLogger).");
    }

    public static synchronized AppLogger getInstance() {
        if (instance == null) {
            instance = new AppLogger();
        }
        return instance;
    }

    public void logInfo(String message) {
        String timestamp = dtf.format(LocalDateTime.now());
        System.out.println("[INFO][" + timestamp + "] " + message);
    }

    public void logError(String message) {
        String timestamp = dtf.format(LocalDateTime.now());
        System.err.println("[ERROR][" + timestamp + "] " + message);
    }

    @Override
    public Object clone() throws CloneNotSupportedException {
        throw new CloneNotSupportedException("Le Singleton AppLogger ne peut pas être cloné !");
    }
}
```

Client.java

```
package variante;

public class Client {
    public static void main(String[] args) {
        AppLogger logger1 = AppLogger.getInstance();
        logger1.logInfo("Démarrage du module de sécurité.");

        AppLogger logger2 = AppLogger.getInstance();
        logger2.logInfo("Connexion au service de paiement établie.");

        System.out.println("\n--- Vérification de l'unicité ---");
        System.out.println("logger1 et logger2 pointent-ils vers la même instance ? " + (logger1 == logger2));
    }
}
```